

INTRODUCTION TO SOFTWARE ENGINEERING — A4 EXAM QUICK SHEET

1. SOFTWARE ENGINEERING

Definition: Application of systematic, disciplined and quantifiable approaches to software development, operation and maintenance.

Software characteristics: intangible; easy to copy; does not physically wear out; changes/evolution can cause degradation; complex; maintenance/evolution is a major part of its life.

Programming vs SE: programming focuses mainly on implementation; SE covers the broader disciplined process, quality, management, documentation, evolution and teamwork.

2. SOFTWARE QUALITY

Quality: degree to which software satisfies stated requirements and expected qualities.

Common quality attributes: correctness, reliability, usability, efficiency, maintainability, portability.

Internal vs External: internal qualities concern structure/code/design; external qualities are observable in operation.

Defect chain: Error (human mistake) → Fault/Defect (problem introduced into software) → Failure (incorrect behavior observed during execution).

Defect handling: prevention → removal → containment.

3. APPLICATION DOMAINS

System software • real-time/embedded systems • business/information systems • engineering/scientific software • AI/data-intensive systems • web/mobile applications.

Legacy software: existing software that continues to be used and often requires maintenance/evolution as environment or requirements change.

4. SOFTWARE PROCESS / LIFECYCLE

Core activities: Communication/Requirements → Planning → Modeling/Design → Construction (coding + testing) → Deployment.

Feedback: lifecycle activities are connected; results from later activities can cause changes to earlier work.

Umbrella activities: project tracking/control, risk management, quality assurance, technical reviews, configuration/change management, documentation, measurement.

5. SOFTWARE ENGINEERING PRINCIPLES

Rigor & formality: use systematic, precise methods.

Separation of concerns: divide a problem into distinct concerns to reduce complexity.

Modularity: decompose system into manageable modules/components.

Abstraction: focus on essential properties while hiding unnecessary detail.

Information hiding: hide internal implementation details behind clear interfaces.

Anticipation of change: design so likely changes can be accommodated.

Generality: prefer solutions applicable to a class of problems, but avoid unnecessary complexity/overengineering.

Incrementality: develop/deliver useful pieces progressively; obtain feedback and reduce risk.

Reuse: use existing components/knowledge where appropriate.

Simplicity: avoid unnecessary complexity.

6. COUPLING & COHESION

Coupling: degree of dependency/interconnection between modules. Aim for **low coupling**.

Coupling terms from lectures: control, common, routine-call, data, content, type-use, stamp, import. (Taxonomies can vary.)

Cohesion: degree to which elements of a module belong together. Aim for **high cohesion**.

Cohesion spectrum: coincidental → logical → temporal → procedural → communicational → sequential → functional.

Rule: High Cohesion + Low Coupling = better modular structure.

7. OPEN-CLOSED PRINCIPLE (OCP)

Open for extension, closed for modification. Add new behavior with minimal changes to stable existing code.

8. KEY EXAM MODELS

Error → Fault → Failure = cause → software defect → observed incorrect behavior.

Process: Requirements → Plan → Design/Model → Code/Test → Deploy → Feedback.

Design goal: abstraction + modularity + information hiding + high cohesion + low coupling + anticipation of change.

9. COMPILER / OTHER DISCIPLINES

Software engineering draws on computer science and related disciplines. Compiler development illustrates requirements, algorithms/data structures, language processing, implementation, testing and tool support working together.

10. RAPID REVISION

Remember: SE ≠ only coding. Quality ≠ only correctness. Maintenance/evolution matters. Requirements drive development. Testing is part of construction. Change is expected. Good modules hide details, depend less on others, and do one coherent job.